

CPP09 — ex02

PmergeMe · Ford–Johnson Merge-Insertion Sort

Deep dive: every type, every function, every design decision

Overview

PmergeMe sorts a sequence of positive integers using the **Ford–Johnson (Merge-Insertion) sort** algorithm — one of the most comparison-efficient sorting algorithms known. It runs the sort twice: once using `std::vector<int>` as the output container and once using `std::deque<int>`, then reports both timings and comparison counts.

The goal of Ford–Johnson is to stay at or very near the information-theoretic lower bound for comparisons: $\lceil \log_2(n!) \rceil$. For $n = 21$ this bound is 66, and our implementation hits it exactly.

File structure

File	Purpose
PmergeMe.hpp	Class and type declarations. FordJohnson template skeleton + PmergeMe class.
PmergeMe.cpp	All method implementations. Template method bodies + PmergeMe methods.
main.cpp	Input validation, creates PmergeMe, triggers sort.
ALGORITHM.md	Extended algorithm documentation.

Data Structures (PmergeMe.hpp — inside FordJohnson)

All internal types are **nested inside the FordJohnson template class**. They are private implementation details not visible outside the class.

STRUCT Block

What it is: The atom of the algorithm. Each input integer becomes one Block stored permanently in the pool.

```
struct Block {
    int value;
    int rep() const { return value; }
};
```

rep() — "representative" — the single value used in all comparisons. In a more general formulation blocks could contain multiple values; `rep()` would then return the maximum. Here each block is always one integer so `rep()` just returns it directly.

Why it exists: Wrapping the integer in a struct lets the algorithm treat elements uniformly through `rep()` and keeps the door open for the growing-block generalisation described in ALGORITHM.md.

```
TYPEDEF BlockId – int
```

An integer index into the `BlockPool`. BlockIds are assigned sequentially (0, 1, 2, ...) as blocks are created and **never change** — they are the stable identity of a block for the entire lifetime of the sort.

Using an integer instead of a pointer means the index is unaffected by any container growth, though in practice the pool is a `std::deque` which guarantees no reallocation anyway.

```
TYPEDEF BlockPool – std::deque<Block>
```

The stable storage for all Block objects. `std::deque` is specifically chosen here because **push_back never invalidates existing elements** — unlike `std::vector` which may reallocate and move everything. Since we hand out integer BlockIds that index into this deque, we need the guarantee that `pool[id]` always points to the same block.

Invariant: `_pool.size()` is fixed for the entire `sort()` call tree. All blocks are created in `run()` before `sort()` is called. `sort()` never calls `makeBlock()`.

```
TYPEDEF Sequence – std::vector<BlockId>
```

An ordered list of block IDs representing a sequence to be sorted or a sorted result. The algorithm operates entirely on Sequences — it shuffles IDs around, never copying actual Block values.

A Sequence passed in may be in any order; a Sequence returned from `sort()` is guaranteed to be ascending by `rep()`.

```
STRUCT PairRecord { BlockId aId, bId; }
```

Records one (winner, loser) pair formed at a single recursion level.

- **aId** — the larger element (the "a" side, winner). Progresses to the next recursion level.
- **bId** — the smaller element (the "b" side, loser). Held in this frame until insertion time.

Lifetime: PairRecords live in a `std::vector<PairRecord> pairs` that is a local variable inside `sort()`. Each recursion depth has its own independent `pairs` vector on the C++ call stack. Deeper frames never touch shallower frames' pairs. This is how pair relationships are preserved without any search.

```
STRUCT PendRecord { BlockId bId, boundId; }
```

Represents a b-element waiting to be inserted into the main chain.

- **bId** — the block to insert.
- **boundId** — the BlockId of bId's a-partner. Used to find the right boundary for the binary search. If `INVALID_ID`, this is an odd element with no bound (searches the entire chain).

The bound is expressed as a BlockId rather than an index because the a-partner's position in the chain *shifts* as earlier insertions push it rightward. Storing the ID lets us always find the current position with `findBoundPos`.

```
STATIC CONST INT INVALID_ID = -1 (file scope in .cpp)
```

Sentinel value meaning "no block". Valid BlockIds are always ≥ 0 (sequential pool indices), so -1 is a safe sentinel. Used for:

- `oddId` when the sequence length is even (no leftover element).
- `boundId` in a `PendRecord` for the odd element (no binary search bound).
- Initialising the `aToB` lookup vector (unfilled slots start as `INVALID_ID`).

FordJohnson — Private Member Variables

```
BlockPool _pool
```

Owns all Block objects for one sort run. Built entirely in `run()` by calling `makeBlock()` once per input integer. Cleared at the start of each new `run()` call so the object can be reused.

```
int _comparisons
```

Running count of key comparisons. Incremented once per `rep() -vs- rep()` test in `makePair()` (pairing comparisons) and once per midpoint test in `binaryInsertPos()` (insertion comparisons). Reset to 0 at the start of each `run()`. Read after sorting via `comparisons()`.

Not counted: The binary search in `findBoundPos()` does not increment `_comparisons` because it is locating a known sorted element, not comparing an unsorted element against a boundary.

FordJohnson — Private Methods

```
PRIVATE BlockId makeBlock(int v)
```

Purpose

Allocates a new Block in the pool with value `v` and returns its stable BlockId.

How it works

```
Block b; b.value = v;
_pool.push_back(b);
return static_cast<BlockId>(_pool.size() - 1);
```

BlockIds are simply the index of the block in the pool. Since blocks are only ever appended (never inserted or removed), the index of a block is permanent.

When called

Only from `run()`, once per input integer, before `sort()` is ever called. Never called from inside the recursive sort.

Complexity

$O(1)$ amortised (`deque::push_back`).

```
PRIVATE PairRecord makePair(BlockId id0, BlockId id1)
```

Purpose

Compares two blocks and returns a PairRecord with the larger as `aId` and the smaller as `bId`. This is the **only place pairing comparisons are counted**.

How it works

```
++_comparisons;
if (_pool[id0].rep() >= _pool[id1].rep())
    return PairRecord(id0, id1); // id0 wins
return PairRecord(id1, id0);    // id1 wins
```

Exactly one comparison per call. Equal elements go to `aId` (arbitrary but deterministic).

When called

In Phase 2 of `sort()`, once per pair of adjacent elements in the sequence.

Complexity

$O(1)$, 1 comparison.

```
STATIC std::vector<int> jacobsthalOrder(int m)
```

Purpose

Generates the order in which the `m` non-odd pend elements should be inserted. Returns a vector of pend indices (0-based) to process in sequence.

The Jacobsthal sequence

Recurrence: $J(k) = J(k-1) + 2 \cdot J(k-2)$, starting 1, 3, 5, 11, 21, 43, 85, ...

In Ford-Johnson terminology, pend element `pend[i]` corresponds to b-element `b_{i+2}` in 1-based notation (`b1` is already in the chain). Group `k` covers 1-based b-indices `J[k]` down to `J[k-1]+1`, inserted **from high to low** within the group.

Example for $m = 4$ (b_2 through b_5)

```
J = [1, 3, 5, ...]
Group 1: b3, b2 → pend indices [1, 0]
Group 2: b5, b4 → pend indices [3, 2]
Result: [1, 0, 3, 2]
```

Why high-to-low within groups

When `b3` is inserted before `b2`, `b2`'s a-partner (`a2`) may shift one position rightward. But Jacobsthal group sizes are chosen so this shift never crosses a power-of-2 boundary — `b2`'s search range cost stays the same.

Safety fallback

After the main loop, any pend index not yet emitted is appended in reverse order. This handles edge cases where the Jacobsthal sequence doesn't cover all indices (should not occur with correct `J` construction, but protects against off-by-one errors).

Complexity

$O(m)$ time and space.

```
PRIVATE int findBoundPos(const Sequence& chain, BlockId boundId) const
```

Purpose

Locates the current position of block `boundId` in the sorted `mainChain`. Returns the index `i` such that `chain[i]` is the a-partner.

How it works

```
int target = _pool[boundId].rep();
int lo = 0, hi = chain.size();
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (_pool[chain[mid]].rep() < target) lo = mid + 1;
    else hi = mid;
}
return lo; // chain[lo].rep() == target
```

Standard lower-bound binary search on `rep()` values. Works because `mainChain` is always sorted ascending by `rep()`.

Why comparisons are NOT counted here

`boundId` is an a-element that the recursion already sorted. Its position in the chain is a bookkeeping fact, not a new comparison between unsorted elements. Counting it would overcount and produce artificially high comparison totals.

Complexity

$O(\log n)$, 0 comparisons counted.

```
PRIVATE int binaryInsertPos(const Sequence& chain, BlockId bId, int bound)
```

Purpose

Finds the correct insertion position for `bId` in `chain[0..bound)`. Every midpoint test increments `_comparisons`.

How it works

```
int target = _pool[bId].rep();
int lo = 0, hi = bound; // search range: [0, bound)
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    ++_comparisons; // ← every test counted
    if (_pool[chain[mid]].rep() < target) lo = mid + 1;
    else hi = mid;
}
return lo;
```

The bound parameter

`bound` is the index returned by `findBoundPos` — the position of `bId`'s a-partner. The b-element is guaranteed to be $\leq$ its a-partner, so it cannot land at or to the right of index `bound`. Excluding that region is the core Ford-Johnson saving: instead of searching a chain of size $2k+1$, we search only k positions.

Complexity

$O(\log \text{bound})$ comparisons, all counted.

```
PRIVATE Sequence sort(const Sequence& seq, int depth = 0)
```

Purpose

The core recursive function. Takes an unsorted Sequence and returns a Sequence of the same IDs sorted ascending by `rep()`.

Every call goes through three phases. The recursion bottoms out at size ≤ 1 .

Base case

```
if (seq.size() <= 1)
    return seq; // trivially sorted, 0 comparisons
```

Phase 2 — Descent (pairing + recursion)

```
// Step 1: pair adjacent elements
std::vector<PairRecord> pairs;
for (size_t i = 0; i + 1 < seq.size(); i += 2)
    pairs.push_back(makePair(seq[i], seq[i+1])); // 1 comparison each

// Step 2: stash odd element (no partner at this depth)
BlockId oddId = (seq.size() % 2 == 1) ? seq.back() : INVALID_ID;

// Step 3: build a-sequence (winners only) and recurse
Sequence aSeq;
for (auto& p : pairs) aSeq.push_back(p.aId);
Sequence sortedA = sort(aSeq, depth + 1); // ← RECURSIVE CALL
```

Key insight: only a-elements recurse. b-elements stay in `pairs[]` on the current stack frame. They are never passed deeper, so their bond with their a-partners can never be lost.

Phase 3 — Unwind (build mainChain and pend)

```
// Build O(1) aId → bId lookup table
std::vector<BlockId> aToB(_pool.size(), INVALID_ID);
for (auto& p : pairs) aToB[p.aId] = p.bId;

// mainChain starts as [b1 | a0 a1 a2 ...]
// b1 goes first for FREE — guaranteed b1 ≤ a0 by pairing
Sequence mainChain;
mainChain.push_back(aToB[sortedA[0]]); // b1
for (auto id : sortedA) mainChain.push_back(id); // a0..an

// pend: remaining b-elements with their a-partner as bound
std::vector<PendRecord> pend;
for (size_t i = 1; i < sortedA.size(); i++)
    pend.push_back(PendRecord(aToB[sortedA[i]], sortedA[i]));
if (oddId != INVALID_ID)
    pend.push_back(PendRecord(oddId, INVALID_ID)); // no bound
```

Why b₁ is free: When (a₀, b₁) was paired, we know b₁ ≤ a₀. After recursion, a₀ is the smallest a-element. Therefore b₁ ≤ a₀ ≤ everything in sortedA. b₁ belongs at position 0 without any comparison.

The aToB vector: BlockIds are consecutive integers [0, pool.size()). Using them as direct array indices gives O(1) lookup with no hashing, no scanning, and no forbidden containers.

Phase 4 — Insertion (Jacobsthal order)

```
// Get Jacobsthal insertion order (excludes the odd element)
int m = pend.size() - (oddId != INVALID_ID ? 1 : 0);
std::vector<int> jOrder = jacobsthalOrder(m);

// Insert each pend element in Jacobsthal order
for (int pi : jOrder) {
    const PendRecord& pr = pend[pi];
    int bound = findBoundPos(mainChain, pr.boundId); // O(log n), uncounted
```

```

    int pos = binaryInsertPos(mainChain, pr.bId, bound); // counted
    mainChain.insert(mainChain.begin() + pos, pr.bId);
}

// Odd element last, no bound – search entire chain
if (oddId != INVALID_ID) {
    int pos = binaryInsertPos(mainChain, pend.back().bId, mainChain.size());
    mainChain.insert(mainChain.begin() + pos, pend.back().bId);
}

return mainChain;

```

Shifting bounds: After inserting a pend element, the a-partners of later elements shift right by 1 in the chain. `findBoundPos` always searches for the current live position, so the bound is always correct regardless of how many insertions have already happened.

Recursion depth

Each recursive call halves the sequence length (only a-elements recurse). Maximum depth is $\lfloor \log_2(n) \rfloor$.

Complexity

Comparisons: $\approx n \cdot \lfloor \log_2 n \rfloor - 0.914n$ (near theoretical minimum).

Data moves: $O(n^2)$ worst case from repeated `vector::insert` shifts.

FordJohnson — Public Methods

PUBLIC FordJohnson() — constructor

Purpose

Initialises `_comparisons` to 0. The pool starts empty; it is filled by `run()` before sorting begins.

```
PUBLIC OutContainer run(const std::vector<int>& input)
```

Purpose

Entry point for one complete sort. Accepts input as `std::vector<int>` regardless of the `OutContainer` template parameter, so both instantiations share one call site. Returns the sorted elements in an `OutContainer`.

Steps

1. Clear pool and reset comparison counter.
2. Call `makeBlock()` once per input integer — builds the pool.
3. Build the initial Sequence (all BlockIds in input order).
4. Call `sort(seq, 0)` — returns sorted Sequence.
5. Flatten: copy `_pool[id].value` into `OutContainer` in sorted order.

Edge cases

- Empty input: returns empty container, 0 comparisons.
- Single element: returns it immediately from base case, 0 comparisons.
- Two elements: one pair, one comparison.

Complexity

$O(n \log n)$ comparisons, $O(n^2)$ data moves.

```
PUBLIC int comparisons() const
```

Purpose

Returns the total number of key comparisons from the most recent `run()`. Used by `sortAndPrint()` to report the count alongside timing.

PmergeMe — Public Interface

The outer class. Owns the raw input data and drives both sort runs.

```
PRIVATE std::vector<int> _vec
PRIVATE std::deque<int> _deq
```

Both containers hold identical copies of the input values. They are only used to report the correct container name and element count in the output line — the actual sorting receives a `std::vector<int>` copy of the input regardless of which output container is being benchmarked.

```
PUBLIC void addValue(int val)
```

Appends `val` to both `_vec` and `_deq`. Called once per validated command-line argument from `main()`.


```
PUBLIC void printBefore() const
```

Prints all values in `_vec` separated by spaces, then a newline. Called once before sorting to show the unsorted input.

```
PUBLIC void sortAndPrint()
```

Purpose

Runs the Ford-Johnson algorithm twice, prints sorted output and timing.

Steps

```
// Both runs receive the same input vector
std::vector<int> input(_vec.begin(), _vec.end());

// Run 1: vector output
clock_t t0 = clock();
FordJohnson<std::vector<int>> fjVec;
std::vector<int> sortedVec = fjVec.run(input);
clock_t t1 = clock();

// Run 2: deque output
clock_t t2 = clock();
FordJohnson<std::deque<int>> fjDeq;
std::deque<int> sortedDeq = fjDeq.run(input);
clock_t t3 = clock();
```

Both runs sort the **same unsorted input** independently so the timings are directly comparable. The two FordJohnson instantiations differ only in the type of container they return — the algorithm inside is identical.

Timing uses `clock()` (CPU time). Divided by `CLOCKS_PER_SEC` and multiplied by 1 000 000 to report microseconds.

Output format (subject requirement):

```
After: 0 1 2 3 ...
Time to process a range of N elements with std::vector : X us (Y comparisons)
Time to process a range of N elements with std::deque : X us (Y comparisons)
```

```
PUBLIC Orthodox Canonical Form
```

Default constructor, copy constructor, copy assignment, destructor — all implemented as required by the 42 norm. Copy and assignment copy both `_vec` and `_deq`. Destructor is trivial (containers self-destruct).

main.cpp — Input Validation

```
int main(int ac, char **av)
```

Validation steps (in order)

1. **Argument count:** at least 2 arguments required (program name + at least one value).
2. **Parse each argument** with `std::stoll(av[i], &pos)`. The `pos` check ensures the entire string was consumed — rejects "123abc".
3. **Range check:** value must be ≥ 0 and $\leq \text{INT_MAX}$.
4. **Duplicate check:** `std::set<int> seen` — `insert` returns false if already present. Duplicates are rejected.
5. Each valid value is passed to `sorter.addValue()`.

```
std::set<int> seen;
for (int i = 1; i < ac; i++) {
    size_t pos;
    long long val = std::stoll(av[i], &pos);
    if (pos != strlen(av[i])) { cerr << "Error"; return 1; }
    if (val < 0 || val > INT_MAX) { cerr << "Error"; return 1; }
    if (!seen.insert((int)val).second) { cerr << "Error"; return 1; }
    sorter.addValue((int)val);
}
```

Any error prints "Error" to stderr and exits with code 1.

Template Design — Why It Works Without a .hpp File

Normally, C++ template method definitions must be visible at the point of instantiation. This usually means putting them in the header. However, our `FordJohnson` template is **only instantiated in `PmergeMe.cpp`**:

```
FordJohnson<std::vector<int>> fjVec;    // instantiation 1
FordJohnson<std::deque<int>>  fjDeq;    // instantiation 2
```

The template method bodies are also in `PmergeMe.cpp` — defined before the instantiations. So when the compiler processes `PmergeMe.cpp` it sees both the declarations (from the header) and the definitions (from the .cpp) in the same translation unit, and can instantiate correctly.

If any other .cpp ever needed to use `FordJohnson`, the definitions would need to move to the header or explicit instantiations would need to be added. For this project, keeping everything in `PmergeMe.cpp` is clean and correct.

Full Call Trace — n = 4, input [3, 1, 4, 2]

```
run([3,1,4,2])
makeBlock(3)→id0 makeBlock(1)→id1 makeBlock(4)→id2 makeBlock(2)→id3
sort([0,1,2,3], depth=0)
Phase 2:
makePair(0,1): pool[0].rep()=3 >= pool[1].rep()=1 → a=0, b=1 (+1 cmp)
makePair(2,3): pool[2].rep()=4 >= pool[3].rep()=2 → a=2, b=3 (+1 cmp)
aSeq = [0, 2] oddId = INVALID_ID
sort([0,2], depth=1)
Phase 2:
makePair(0,2): pool[0].rep()=3 < pool[2].rep()=4 → a=2, b=0 (+1 cmp)
aSeq = [2]
sort([2], depth=2) → BASE CASE → return [2]
Phase 3:
aToB[2]=0
mainChain = [0, 2] (b1=id0=val3, a0=id2=val4)
```

```

    pend = [] (only one pair, b1 already placed)
    Phase 4: nothing to insert
    return [0, 2] ← sorted by rep: val3, val4
Phase 3 (depth=0):
    sortedA = [0, 2]
    aToB[0]=1, aToB[2]=3
    mainChain = [1, 0, 2] (b1=id1=val1, then a0=val3, a1=val4)
    pend = [{bId=3, boundId=2}] (val2, bound=val4)
Phase 4:
    jOrder = jacobsthalOrder(1) = [0]
    Insert pend[0]: bId=3 (val2), boundId=2 (val4)
    findBoundPos(chain=[1,0,2], boundId=2): target=4, returns index 2
    binaryInsertPos(chain, bId=3, bound=2):
        lo=0,hi=2 → mid=1, pool[chain[1]].rep()=3 > 2 → hi=1 (+1 cmp)
        lo=0,hi=1 → mid=0, pool[chain[0]].rep()=1 < 2 → lo=1 (+1 cmp)
        return 1
    insert id3 at pos 1: chain = [1, 3, 0, 2]
    return [1, 3, 0, 2] ← val1, val2, val3, val4 ✓

Total comparisons: 5 (lower bound ⌈log2(4!)⌋ = 5) ✓

```